Module 7 - Advanced SQL Queries: Aggregates, Subqueries & Views

Learning Outcomes

By the end of this module, you will be able to:

1. Use aggregate functions to summarize data.
2. Group and filter results using `GROUP BY` and `HAVING`.
3. Write subqueries within other queries.
4. Create and use views for abstraction and security.
5. Explain the purpose of indexes in query optimization.

From Retrieval to Insight

Previously (Module 6):

- Basic `SELECT`, `WHERE`, `ORDER BY`, `JOIN`
- Retrieving detailed data

Now: Move from *retrieval* ➜ *insight* using summarization, nesting, and abstraction.

When Queries Become Questions

"How many?"
"What's the average?"
"Who hasn't submitted yet?"

SQL can now:

- Combine rows into summaries
- Query the results of other queries
- Save & reuse logic using **views**

Part 1 – Aggregate Functions

What Are Aggregate Functions?

RDBMS works on **sets of data**, but it seems to us that the processing is done one row at a time.

For some questions you need to work with **collections of rows as a single unit**

- Perform calculations across a set of rows ➜ return a single value.

This is done with **aggregate functions**

- count
- min / max
- avg
- ...

Aggregate Functions

- Mathematical Summaries
- Most commonly used in SELECT

Function	Purpose	Example
`COUNT()`	Count rows	`COUNT(*)`
`SUM()`	Add totals	`SUM(price)`
`AVG()`	Average	`AVG(grade)`
`MIN()`	Smallest	`MIN(price)`
`MAX()`	Largest	`MAX(price)`

Example: Counting Products

```sql
SELECT COUNT(*) AS total_products
FROM product;
```

Result: One row showing total number of products.

CountOfP_CODE
16

Filtering Aggregates

```sql
SELECT COUNT(*) AS cheap_products
FROM product
WHERE p_price < 10;
```

Counts only products below $10.

COUNT vs COUNT(*)

Query	What It Counts	
`COUNT(*)`	All rows	CountOfP_CODE: 16
`COUNT(column)`	Only non-null values	CountOfV_CODE: 14

Example:

```
SELECT COUNT(v_code), COUNT(*)
FROM product;
```

Counting DISTINCT Values

`COUNT(DISTINCT column)` counts **unique values** instead of all rows.

```
SELECT COUNT(DISTINCT v_code) AS "Count Distinct"
FROM product;
```

Result: Number of distinct vendors in the product table.

Count Distinct
6

- FROM retrieves all values
- DISTINCT removes duplicate values
- COUNT tallies the non-null values

MIN & MAX Functions

Find smallest or largest values in a column.

```
SELECT MAX(p_price) AS MAXPRICE, MIN(p_price) AS MINPRICE
FROM product;
```

Result: Shows price range of products.

MAXPRICE	MINPRICE
256.99	4.99

SUM Function

```
SELECT SUM( CUS_BALANCE) AS TOTALBALANACE
FROM CUSTOMER ;
```

Total amount owned by your customers

```
SELECT SUM( P_QOH * P_PRICE) AS TOTALVALUE
FROM PRODUCT ;
```

Total value of all items carried in inventory

Derived values & **formulas** are acceptable

TOTVALUE
15084.52

AVG Function

```
SELECT AVG( P_PRICE) AS AVGPRICE
FROM PRODUCT ;
```

AVGPRICE
56.42125

Part 2 – Grouping Data

Why GROUP BY?

Summarize **by category** instead of entire table.

```
SELECT v_code, COUNT(*) AS num_products
FROM product
GROUP BY v_code;
```

Result: Product count per vendor.

How GROUP BY Treats NULL Values

```
SELECT v_code, AVG(P_PRICE) AS AVGPRICE
FROM product
GROUP BY v_code;
```

When the grouping column contains NULLs, SQL treats all NULLs as one single group.

V_CODE	AVGPRICE
	10.13
21225	8.47
21231	8.45
21344	12.49
23119	41.97
24288	155.59
25595	89.63

Multiple Group Columns

```sql
SELECT term_code, set_code, COUNT(*) AS num_students
FROM enrollment
GROUP BY term_code, set_code;
```

Counts students by **term** and **set**.

GROUP BY Rules

- Columns in SELECT must be in GROUP BY *or* in an aggregate.
- Each group ➜ one result row.

❌ Invalid:

```
SELECT v_code, v_name, COUNT(p_code)
FROM product
GROUP BY v_code;
```

(v_name missing from GROUP BY)

Filtering Groups with HAVING

`WHERE` filters rows ➜ `HAVING` filters groups.

```
SELECT v_code, AVG(p_price) AS avg_price
FROM product
GROUP BY v_code
HAVING AVG(p_price) > 50;
```

WHERE vs HAVING

Possible to have both WHERE and HAVING in one query

WHERE executes **before grouping**

- Applies to rows
- cannot contain aggregate functions

HAVING is executed **after grouping**

- Applies to groups
- either keeps or eliminates the entire group (not possible to restrict some rows in a group and leave others)
- can contain and almost always contains aggregate functions

SQL Execution Order

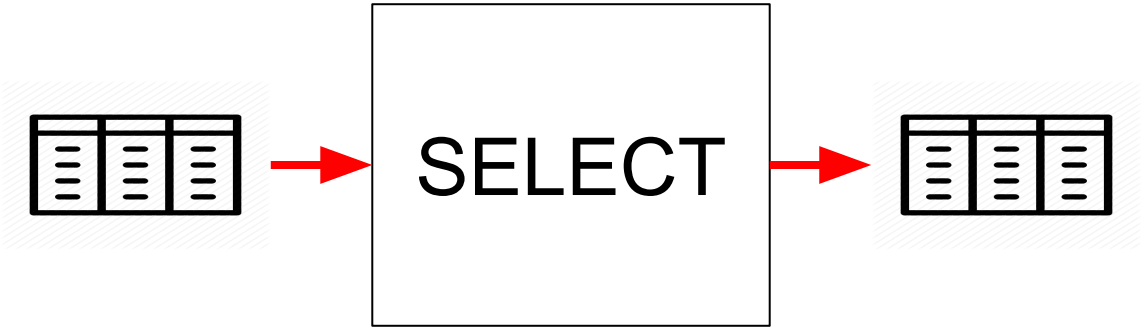
```
SELECT  columnlist
FROM    tablelist
[ WHERE     conditionlist ]
[ GROUP BY  columnlist ]
[ HAVING    conditionlist ]
[ ORDER BY  columnlist [ASC | DESC] ];
```

1. FROM → 2. WHERE → 3. GROUP BY → 4. HAVING → 5. SELECT → 6. ORDER BY

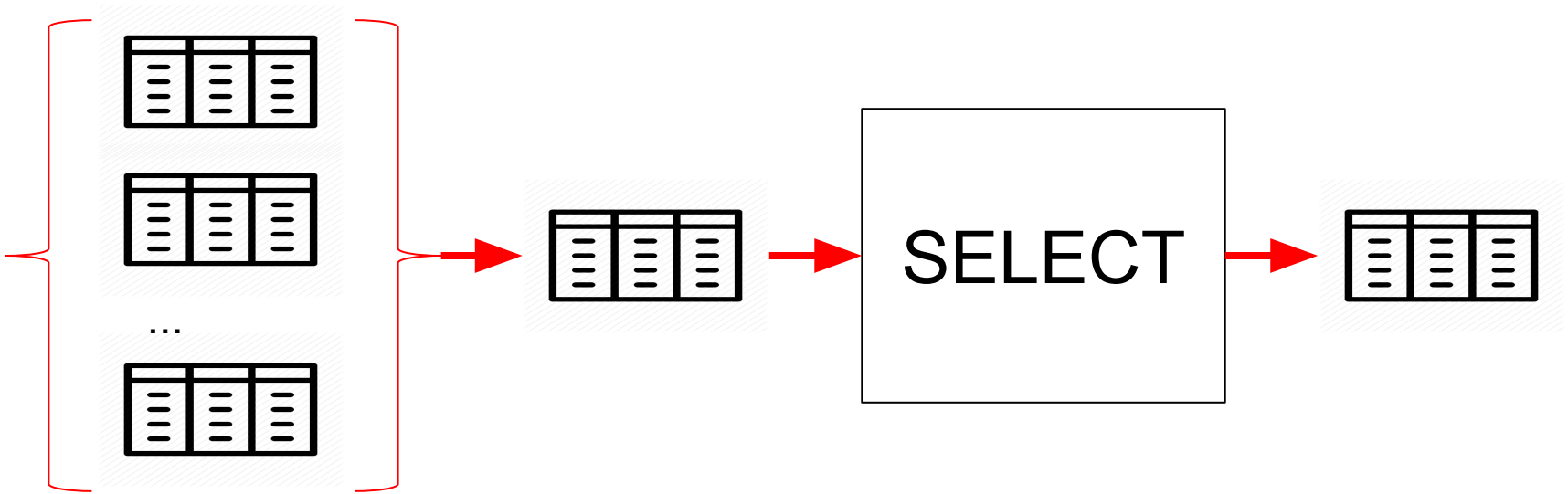
💡 Think: SQL reads top-down but executes inside-out.

Part 3 - Subqueries

SELECT expects an **input table** and always returns an **output table**.



Sometimes you need to **join** multiple table to form the input table



Example

You want a list of all vendors that **do not provide** any products.

- Vendor table includes the vendor information

V_CODE	V_NAME	V_CONTACT

- Product table includes V_CODE FK which specifies what vendor has provided each product

P_CODE	P_DESCRIPT	P_PRICE	V_CODE

Example: Forming the Input Table with JOIN

V_CODE	V_NAME	V_CONTACT

<-- LEFT JOIN -->

P_CODE	P_DESCRIPT	P_PRICE	V_CODE

Input Table

V_CODE	V_NAME	V_CONTACT	P_CODE	P_DESCRIPT	P_PRICE

Input Table

V_CODE	V_NAME	V_CONTACT	P_CODE	P_DESCRIPT	P_PRICE

```
SELECT P_CODE, VENDOR.V_CODE, V_NAME
FROM VENDOR LEFT JOIN PRODUCT ON VENDOR.V_CODE = PRODUCT.V_CODE
WHERE P_CODE IS NULL;
```

Result Table

P_CODE	V_CODE	V_NAME

Digging Deeper - A Second Approach

Query:

Get a list of all vendors that **do not provide** any products.

Solution # 2:

If we knew all vendors who ***do provide*** products currently (*CURRENT_VEDNORS*), then we could check against this list of current vendors and find vendors who *do **not** provide* any products:

```
SELECT V_CODE, V_NAME
FROM VENDOR
WHERE V_CODE  NOT IN (CURRENT_VEDNORS);
```

How do we generate the list of current vendors?

P_CODE	P_DESCRIPT	P_PRICE	V_CODE
			ABC
			DEF

```
SELECT V_CODE
FROM PRODUCT
WHERE V_CODE IS NOT NULL;
```

V_CODE
ABC
DEF

CURRENT VENDORS

P_CODE	P_DESCRIPT	P_PRICE	V_CODE
			ABC
			DEF

```
SELECT V_CODE
FROM PRODUCT
WHERE V_CODE IS NOT NULL;
```

V_CODE
ABC
DEF

V_CODE	V_NAME	V_CONTACT
ABC		
DEF		
GHI		

```
SELECT V_CODE, V_NAME
FROM VENDOR
WHERE V_CODE NOT IN (
);
```

V_CODE
ABC
DEF

CURRENT VENDORS

CURRENT VENDORS

V_CODE	V_NAME
GHI	

P_CODE	P_DESCRIPT	P_PRICE	V_CODE
			ABC
			DEF

V_CODE	V_NAME	V_CONTACT
ABC		
DEF		
GHI		

```
SELECT V_CODE
FROM PRODUCT
WHERE V_CODE IS NOT NULL;
```

```
SELECT V_CODE, V_NAME
FROM VENDOR
WHERE V_CODE NOT IN (
    SELECT V_CODE
    FROM PRODUCT
    WHERE V_CODE IS NOT NULL;
);
```

V_CODE
ABC
DEF

CURRENT VENDORS

V_CODE	V_NAME
GHI	

Introducing Subqueries

V_CODE	V_NAME	V_CONTACT
ABC		
DEF		
GHI		

```
SELECT V_CODE, V_NAME
FROM VENDOR
WHERE V_CODE NOT IN (
    SELECT V_CODE
    FROM PRODUCT
    WHERE V_CODE IS NOT NULL;
);
```

- This is called a **subquery!**
- A query inside another query
- Also called an **inner query**

V_CODE	V_NAME
GHI	

Introducing Subqueries

V_CODE	V_NAME	V_CONTACT
ABC		
DEF		
GHI		

Outer query

```
SELECT V_CODE, V_NAME
FROM VENDOR
WHERE V_CODE NOT IN (
    SELECT V_CODE
    FROM PRODUCT
    WHERE V_CODE IS NOT NULL;
);
```

- This is called a **subquery!**
- A query inside another query
- Also called an **inner query**

V_CODE	V_NAME
GHI	

Introducing Subqueries

V_CODE	V_NAME	V_CONTACT
ABC		
DEF		
GHI		

```
SELECT V_CODE, V_NAME
FROM VENDOR
WHERE V_CODE NOT IN (
    SELECT V_CODE
    FROM PRODUCT
    WHERE V_CODE IS NOT NULL;
);
```

Inner query gets **executed first**

V_CODE	V_NAME
GHI	

Introducing Subqueries

V_CODE	V_NAME	V_CONTACT
ABC		
DEF		
GHI		

Outer query

```
SELECT V_CODE, V_NAME
FROM VENDOR
WHERE V_CODE NOT IN (
);
```

V_CODE
ABC
DEF

CURRENT VENDORS

The output of inner query becomes the **input for the outer query.** (And since the subquery is generating a column, we call this subquery a **list subquery**)

V_CODE	V_NAME
GHI	

Example 2: List Subquery

```sql
SELECT student_id, first_name
FROM student
WHERE student_id NOT IN (
  SELECT student_id
  FROM submission
  WHERE lab_id = 'L1'
);
```

Finds students without Lab 1 submissions.

Example 3: Scalar Subquery

Query:

Get the list of all products that are more expensive than the avg price

Solution:

If we knew the average price (*AVG_PRICE*), then we could select all such products:

```
SELECT P_CODE, P_PRICE
FROM PRODUCT
WHERE P_PRICE  > (AVG_PRICE);
```

How do we get the average price of all products?

P_CODE	P_DESCRIPT	P_PRICE	V_CODE
		12	
		34	
		14	

```
SELECT AVG(P_PRICE)
FROM PRODUCT ;
```

AVG(PRICE)
20

AVG PRICE

P_CODE	P_DESCRIPT	P_PRICE	V_CODE
		12	
		34	
		14	

```
SELECT AVG(P_PRICE)
FROM PRODUCT ;
```

AVG(PRICE)
20

AVG PRICE

P_CODE	P_DESCRIPT	P_PRICE	V_CODE
		12	
		34	
		14	

```
SELECT P_CODE, P_PRCE
FROM PRODUCT
WHERE P_PRICE > (
```

AVG(PRICE)
20

AVG PRICE

```
);
```

P_CODE	P_PRICE
	34

P_CODE	P_DESCRIPT	P_PRICE	V_CODE
		12	
		34	
		14	

```
SELECT AVG(P_PRICE)
FROM PRODUCT ;
```

AVG(PRICE)
20

AVG PRICE

P_CODE	P_DESCRIPT	P_PRICE	V_CODE
		12	
		34	
		14	

```
SELECT P_CODE, P_PRCE
FROM PRODUCT
WHERE P_PRICE > (
    SELECT AVG(P_PRICE)
    FROM PRODUCT ;
);
```

P_CODE	P_PRICE
	34

We use subqueries to **generate unknown information** that is needed in a query.

Subqueries in IN & WHERE Clauses

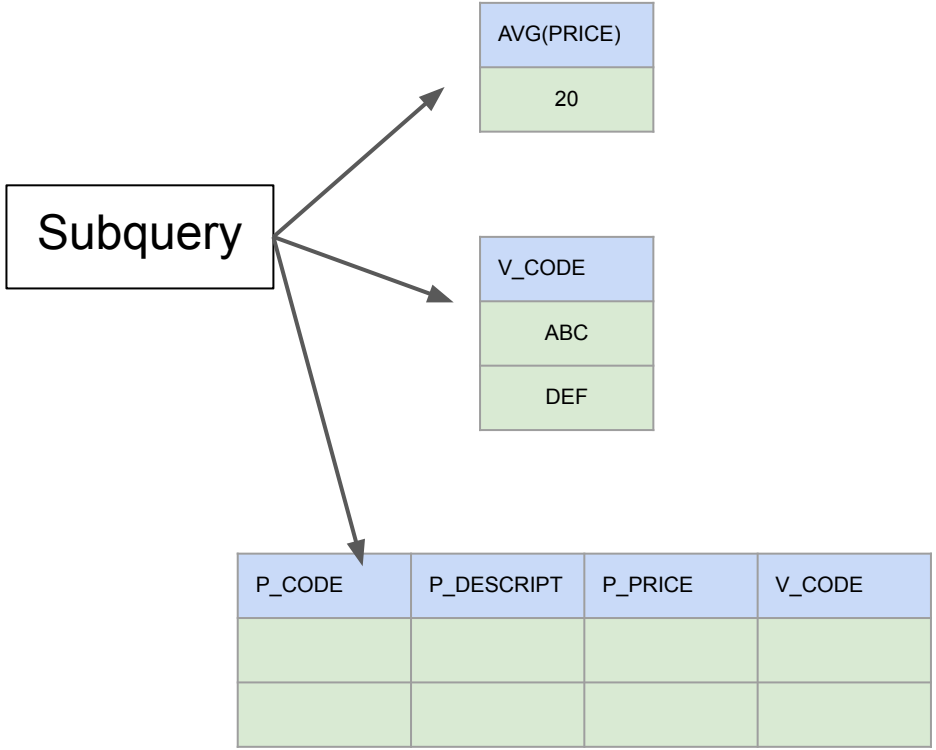
```
SELECT V_CODE, V_NAME
FROM VENDOR
WHERE V_CODE NOT IN (CURRENT_VEDNORS);
```

In the **first example**, *CURRENT_VENDORS* is the unknown but needed information that we can generate with a **list subquery**

```
SELECT P_CODE, P_PRICE
FROM PRODUCT
WHERE P_PRICE > (AVG_PRICE);
```

In the **second example** *AVG_PRICE* is the unknown but needed information that we generate with a **scalar subquery**

Output of a Subquery



One single value

- One column & one row
- Used anywhere a single value is expected (e.g in comparison expressions)

A list of values

- One column / Multiple rows
- Used anywhere a list of values is expected (e.g IN clause)

A virtual table

- Multicolumn, Multirow
- Used anywhere a table is expected (e.g. with FROM clause)

Example 4: Table Subquery

```sql
SELECT vendor_name, total_value
FROM (
  SELECT v_code, SUM(p_qoh * p_price) AS total_value
  FROM product
  GROUP BY v_code
) AS vendor_totals
JOIN vendor USING (v_code);
```

Aggregates and joins subquery results.

Subquery Types Summary

Type	Returns	Used In	Example
Scalar	One value	WHERE	`> (SELECT AVG(...))`
List	One column	IN / NOT IN	`IN (SELECT id ...)`
Table	Multiple columns	FROM	`FROM (SELECT ...)`

Subquery Tips

- Write/test inner query first.
- Ensure matching data types.
- Alias subqueries.
- Use subqueries for logic clarity; joins for speed.

Part 4 - Views

What is a View?

A **virtual table** based on a SELECT query.

- You can treat a view **like a table**
- The data values in the view always reflect **live data** from the **base tables** referenced in the query
- The select **query gets stored** in the database
- Every time a view is provoked, its data values are **dynamically re-created / updated** on demand
- Useful for **access abstraction**

Creating a View

```
CREATE VIEW active_students AS
SELECT s.student_id, s.first_name, s.last_name
FROM student s
JOIN enrollment e USING (student_id)
WHERE e.status = 'active';
```

Querying a View

```
SELECT * FROM active_students;
```

Behaves like a real table.

Why Use Views?

- ✅ Simplify complex queries
- ✅ Provide data security
- ✅ Standardize reporting logic
- ✅ Support consistent interfaces

Updating Through Views

- Single-table views may be updatable.
- Multi-table views usually read-only.
- Use for simplicity, not frequent writes.

Dropping a View

```
DROP VIEW active_students;
```

Removes view definition.

Part 5 – Indexes

What is an Index?

- An index works **similar to a library catalogue**
- An index consists of an **index key** and a **set of pointers** to table rows
- This arrangement of (key, pointers) helps us access table rows more quickly

Example - Indexing the Painting Table

FIGURE 3.18 THE IMPLEMENTED 1:M RELATIONSHIP BETWEEN PAINTER AND PAINTING

Table name: PAINTER
Primary key: PAINTER_NUM
Foreign key: none

Database name: Ch03_Museum

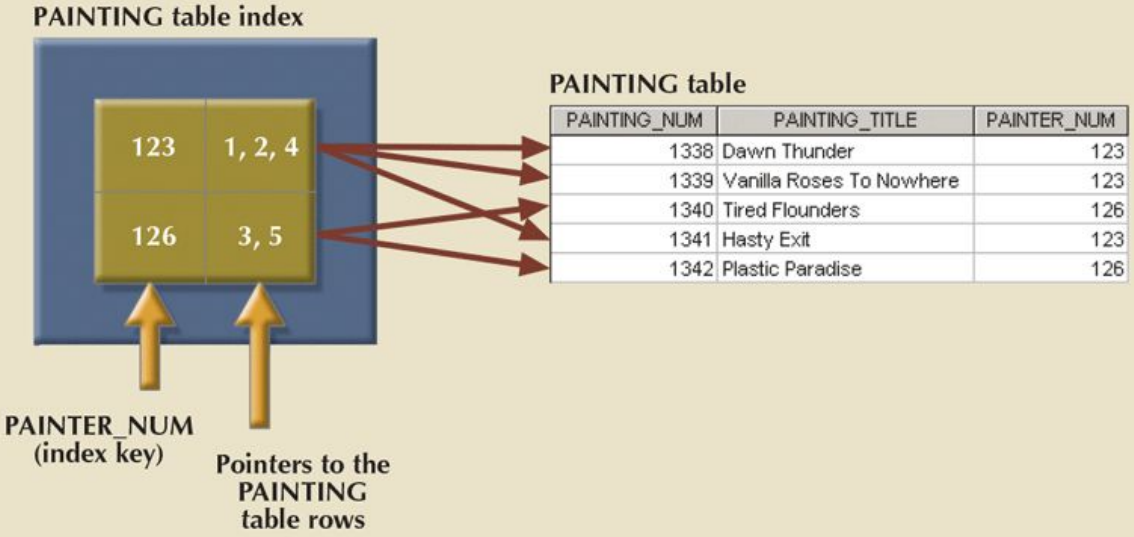
PAINTER_NUM	PAINTER_LNAME	PAINTER_FNAME	PAINTER_INITIAL
123	Ross	Georgette	P
126	Itero	Julio	G

Table name: PAINTING
Primary key: PAINTING_NUM
Foreign key: PAINTER_NUM

PAINTING_NUM	PAINTING_TITLE	PAINTER_NUM
1338	Dawn Thunder	123
1339	Vanilla Roses To Nowhere	123
1340	Tired Flounders	126
1341	Hasty Exit	123
1342	Plastic Paradise	126

Example - Indexing the Painting Table

FIGURE 3.31 COMPONENTS OF AN INDEX



PAINTING_NUM	PAINTING_TITLE	PAINTER_NUM
1338	Dawn Thunder	123
1339	Vanilla Roses To Nowhere	123
1340	Tired Flounders	126
1341	Hasty Exit	123
1342	Plastic Paradise	126

Where to use indexes?

- Common Practice: Create an index on any field that is used as a **search key**
  - In comparisons done in conditional expressions
  - In Sorting
- Each index is associated only with one table
- A table can have many indexes
- PKs are automatically indexed

CREATE INDEX Command

CREATE [UNIQUE] INDEX *indexname* ON *tablename*(*column1*[, *column2*])

- Notice that UNIQUE is optional
- If **unique** is used, not only an index is created to speed up searches, but also duplicate values are avoided
- Notice that it is possible to create indexes on multiple attributes → **Composite Index**

Example: CREATE Index to speed up searches

Query:

List all customers living in a specific city

Index:

Since we are querying the CUSTOMER table by city, we will create an index on city column to speed up the search

CREATE INDEX *CUS_CITYX* **ON** *CUSTOMER* **(**CUS_CITY**)**

Example: CREATE UNIQUE Index to enforce constraints

Constraint:

An employee can take a test only once on a given date.

Index:

By creating a unique index we can ensure the constraint is enforced

CREATE UNIQUE INDEX *EMP_TESTDEX* **ON** *TEST* **(**EMP_NUM, TEST_CODE, TEST_DATE**)**

Example: CREATE Index to retrieve data in a specified order

Preferred Order:

When returning products, return the most expensive product first

Index:

CREATE INDEX *PROD_PRICEX* **ON** *PRODUCT* **(**P_PRICE **DESC)**

DROP INDEX Command

DROP INDEX *indexname*

Summary

Concept	Purpose	Key Syntax
Aggregate	Summarize rows	`SUM`, `COUNT`, `AVG`
GROUP BY	Group results	`GROUP BY col`
HAVING	Filter groups	`HAVING condition`
Subquery	Nested query	`(SELECT …)`
View	Virtual table	`CREATE VIEW`
Index	Optimize search	`CREATE INDEX`